

Towards Automatic Dominance Breaking

Christopher Mears, Maria Garcia de la Banda

Faculty of IT, Monash University, Australia
 {chris.mears,maria.garciadelabanda}@monash.edu

Abstract. Constraint optimisation problems can exhibit dominance relations whose exploitation can lead to dramatic reductions in search space. We propose an automatic method to detect some of the dominance relations identified recently by Chu and Stuckey [1] for a given optimisation problem, and to construct the associated dominance breaking constraints. Experimental results show that the method is able to find different kinds of dominance relations and generate effective dominance breaking constraints.

1 Introduction

Dominance relations can be seen as a generalisation of the well-known symmetry and conditional symmetry relations for optimisation problems. Thus, dominance breaking offers similar or even greater opportunities for pruning the search space than symmetry breaking. Interestingly, not much work has been devoted to dominance relations. While there are several well-known methods to identify and exploit most kinds of symmetry relations, the identification and exploitation of dominance relations has followed a problem-specific treatment (e.g. [3, 11, 12]) that has offered little insight into possible generalisations.

Recently, Chu and Stuckey [1] proposed a generic method for manually identifying a large class of dominance relations and manually exploiting them by using dominance breaking constraints. Intuitively, the method aims at finding mappings σ that, under a certain condition *cond*, are known to map solutions to better solutions. That is, if *cond* holds, $\sigma(\theta)$ maps solution θ to another solution, whose objective function value $f(\sigma(\theta))$ is better (smaller/greater) than that of $f(\theta)$. Then, the negation of *cond* can be used as a dominance breaking constraint. Chu and Stuckey [1] observe that a dominance mapping is good if its associated dominance breaking constraint $\neg \text{cond}$ is simple, as simpler constraints will propagate faster and prune more. Clearly, mappings that are symmetries will lead to a simpler constraint, as they always map solutions to solutions and, therefore, *cond* only needs to ensure that $f(\sigma(\theta))$ is better.

As a result we can increase the usefulness of Chu and Stuckey's work by automating this process, that is, by developing a method to (a) automatically identify symmetries for a given problem, and (b) automatically construct the associated dominance breaking constraints. Note that these dominance breaking constraints are not symmetry breaking constraints, as the key element is for $f(\sigma(\theta))$ to be better. Further, the symmetries need to be detected for the inherent satisfaction problem — that is, the optimisation problem without the objective function — because otherwise $f(\sigma(\theta))$ must be equal to $f(\theta)$, and cannot be better than it.

Our first attempt at automating the process focused on ensuring the automatic inference of symmetries was done very efficiently, as it needs to be done every time we search for a solution to the problem. However, the methods that were efficient were not capable of inferring useful symmetries. We thus decided to try more accurate methods that could infer symmetries for the problem *model*, that is, a parameterised version of the problem specification that can later be instantiated with particular input data for the parameters. While these methods can take significant time, they are scalable and, once applied, the symmetries inferred can be used for all instances of the model, thus compensating for the extra time required.

While we obtained good results using the symmetries automatically inferred for the model, we realised that more could be done as not all dominance mappings are actual symmetries. Interestingly, we have determined that some mappings associated with the dominance breaking constraints identified in [1] correspond to *almost symmetries* [5], that is, to mappings that are symmetries of the problem if a few constraints are ignored. Therefore, we have extended our automatic detection process to infer dominance constraints derived from almost symmetries. In particular, given the importance of obtaining a simple dominance constraint, we have focused on the class of almost symmetries that are obtained from eliminating a single constraint from the problem. Our experimental evaluation shows that the automatically generated dominance breaking constraints can yield significant improvement in search performance, particularly when some of our currently manual refinements become automated.

2 Background

This section provides the necessary background notation and concepts, following mainly the definitions given in [1]. Let $\equiv$ denote syntactical identity. A *variable* v is a mathematical quantity capable of assuming any value from its *initial domain*, which is determined by its type. Given a set of variables V , let Θ_V denote the set of valuations over V where each variable in V is assigned to a value in its initial domain. A *constraint* c over V is defined by a set of valuations $\text{solns}(c) \subseteq \Theta_V$. Given a valuation θ over $V' \supset V$, we say θ *satisfies* c if the restriction of θ onto V is in $\text{solns}(c)$. Otherwise, we say that θ *violates* c . A *domain* D over variables V is a set of unary constraints, one for each variable in V . In an abuse of notation, if a symbol A refers to a set of constraints $\{c_1, \dots, c_n\}$, we will often also use A to refer to the constraint $c_1 \wedge \dots \wedge c_n$.

A *Constraint Satisfaction Problem* (CSP) is a tuple $P \equiv (V, D, C)$, where V is a set of variables, D is a domain over V , and C is a set of n -ary constraints, each $c \in C$ defined over a subset of V . A valuation θ over V is a solution of P if it satisfies every constraint in D and C . A *Constraint Optimisation Problem* (COP) $P \equiv (V, D, C, f)$ extends a CSP by adding an objective function f mapping v to an ordered set, e.g., $\mathbb{Z}$ or $\mathbb{R}$, with the aim of finding a solution to the CSP that minimises/maximises f . As for [1], we deal with finite domain problems only, i.e., where the initial domain D constrains each variable to take values from a finite set of values. Further, for brevity, we assume that all objective functions are to be minimised, and consider constraint satisfaction problems as constraint optimisation problems where $f(\theta) = 0$ for any valuation θ .

We will represent CSPs and COPs using the MiniZinc [10] modelling language. Our automatic detection method follows [8] in distinguishing between a CSP (or COP) *model* and its *instance*. A *model* is any specification that can be expressed as a MiniZinc program with at least one unspecified parameter, where a parameter can be an integer, a set of integers or a sequence of integers. An *instance* of a model (the actual CSP or COP) is then defined as the result of extending the model by providing values to all its parameters. Therefore, an instance also corresponds to a MiniZinc program, but one without unspecified parameters.

Dominance relations for a COP are defined as follows.

Definition 1. (from [1].) A dominance relation $\prec$ for a COP $P \equiv (X, D, C, f)$ is a transitive and irreflexive binary relation on Θ_X such that if $\theta_1 \prec \theta_2$, then either: 1) θ_1 is a solution and θ_2 is a non-solution, or 2) they are both solutions or both non-solutions and $f(\theta_1) \leq f(\theta_2)$.

If $\theta_1 \prec \theta_2$, we say that θ_1 dominates θ_2 . As proved in [1], any dominated valuation can be safely discarded. This can be extended from valuations to search nodes, so that we can also discard dominated search nodes.

Definition 2. (from [1].) Let D_1 and D_2 be the domains from two different search nodes. If $\forall \theta_2 \in \text{sols}(D_2), \exists \theta_1 \in \text{sols}(D_1)$ s.t. $\theta_1 \prec \theta_2$, then we define $D_1 \prec D_2$.

Chu and Stuckey [1] provide a method for identifying dominance relations in a COP and adding constraints to exploit them. Formally, the method has four steps [1]:

1. Find mappings $\sigma : \Theta_X \rightarrow \Theta_X$ that are likely to map solutions to *better* solutions, that is, mappings for which condition $\sigma(\theta) \prec \theta$ often holds.
2. For each σ , find a constraint $\text{scond}(\sigma)$ s.t. if $\theta \in \text{sols}(C \wedge D \wedge \text{scond}(\sigma))$, then $\sigma(\theta) \in \text{sols}(C \wedge D)$, i.e., a constraint under which σ maps solutions to solutions.
3. For each σ , find a constraint $\text{ocond}(\sigma)$ s.t. if $\theta \in \text{sols}(C \wedge D \wedge \text{ocond}(\sigma))$, then $f(\sigma(\theta)) < f(\theta)$, i.e., a constraint under which σ maps solutions to better valuations.
4. For each σ , post the dominance breaking constraint $\neg(\text{scond}(\sigma) \wedge \text{ocond}(\sigma))$.

A *solution symmetry* of CSP $P \equiv (V, D, C)$ is a permutation of the variable/value pairs in P that maps solutions of P to solutions of P and non-solutions of P to non-solutions of P [2]. A special case of solution symmetry, called a *valuation symmetry* in [5] (and syntactic symmetry in [9]), occurs when σ maps all the variable/value pairs of each variable to the variable/value pairs of another variable. Importantly, such symmetries always map a valuation to another valuation and, thus, they can easily be extended to map constraints to constraints, since the mapping on variables is uniform. For non-valuation symmetries this definition may not be meaningful since $\sigma(\theta)$ may not be a valuation (e.g., might contain two different values for the same variable). Many common symmetries are valuation symmetries, including interchangeable variables, interchangeable values, or interchangeable rows or columns in matrix problems [6].

A permutation group is a set of permutations that is closed under composition and inverses. The solution (and valuation) symmetries of a CSP P form a permutation group, where each element is a permutation on the set of variable/value pairs of

P . Given permutations $\{f_1, f_2, \dots, f_n\}$ for P , we denote by $[f_1, f_2, \dots, f_n]$ the closure under composition and inverses of $\{f_1, f_2, \dots, f_n\}$. For a permutation group G , if $[f_1, f_2, \dots, f_n] = G$ then we say that $\{f_1, f_2, \dots, f_n\}$ *generates* G , and we call $\{f_1, f_2, \dots, f_n\}$ a *generating set* of G .

3 Automatically inferring almost symmetries

The input to our method is a MiniZinc COP model $Mzn \equiv (X, D, C, f)$ and n MiniZinc input data files $d_1, \dots, d_n$. Given this, our method first automatically infers symmetries and almost symmetries of Mzn . Then, it uses some of these symmetries as dominance mappings to generate dominance breaking constraints that are added to Mzn . This section focuses on the first task, which is achieved by following the steps shown in Figure 1.

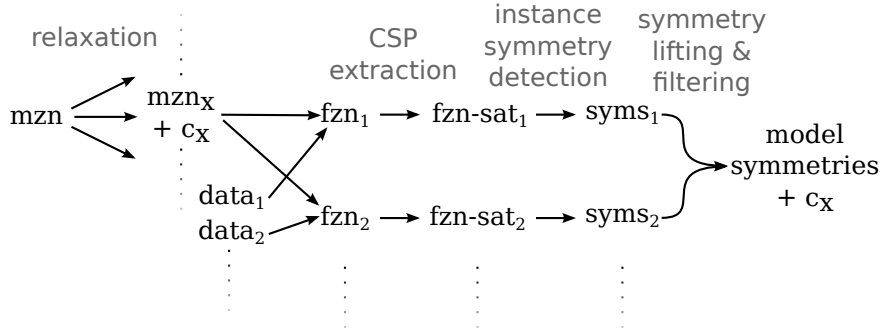


Fig. 1. Symmetry detection overview.

Reading Figure 1 from the left, the first step takes the original MiniZinc model Mzn and relaxes it by removing each of its constraints separately, each time giving a relaxed model paired with the removed constraint. Next, the (relaxed) model is compiled using each data file, giving several FlatZinc files. Each FlatZinc file has its satisfaction core extracted, and the symmetries found for that core. Finally, the instance symmetries are lifted to the model level, so that the final result is the set of symmetries for the (relaxed) model, paired with the constraint removed at the beginning.

Each of these steps is explained in the following subsections.

3.1 Generating relaxed versions of a COP model

The first step of our method is to obtain relaxed versions of the MiniZinc COP model Mzn by simply removing some of its constraints. Our aim is to use these relaxed versions to find almost symmetries of Mzn , that is, symmetries of the relaxed versions that are not symmetries of Mzn . Like any regular symmetry, an almost symmetry σ is a mapping that can be used to generate the dominance breaking constraint

$scond(\sigma) \rightarrow \neg ocond(\sigma)$). The main difference is that for a symmetry the $scond(\sigma)$ component is trivially true, while for an almost symmetry the $scond(\sigma)$ component is the mapped version of the constraint(s) removed to obtain the relaxed version of Mzn . Note that these almost symmetries might be very useful if they lead to simple conditions for producing better solutions.

We have automated the generation of relaxed problems by using the structure present in any MiniZinc model, that is, their set of constraint “items”, which typically correspond to the high-level logical constraints of the problem. Assuming our input COP model $Mzn \equiv (X, D, C, f)$ has m constraint items, we generate a set of relaxed problems $\{Mzn_1, \dots, Mzn_m\}$, where $Mzn_i \equiv (X, C_i, D, f)$ and C_i is obtained by removing constraint item c_i from C .

Example 1. The following model for the blackhole solitaire problem (see Section 5.2) has four constraints, denoted in the model as A, B, C and D:

```

1  % Card at position
2  array[1..52] of var 1..52: cardAt;
3  % Position of card
4  array[1..52] of var 1..52: position;
5
6  % Constraint A: Ace of Spades is first card
7  constraint cardAt[1] == 1;
8
9  % Constraint B: Consecutive cards match
10 constraint forall(i in 1..51)
11     ( table([cardAt[i], cardAt[i+1]], neighbours) );
12
13 % Constraint C: Link card-at and position
14 constraint inverse(cardAt, position) :: domain;
15
16 % Constraint D: A card must be played before the one under it.
17 constraint forall(i in 1..17, j in 1..2)
18     ( position[layout[i,j]] < position[layout[i,j+1]] );

```

Four relaxed models can thus be derived from it: a model with only constraints A,B,C; one with A,B,D; one with A,C,D; and one with B,C,D.

For each relaxed problem Mzn_i , we remember the constraint item c_i that was removed from Mzn to obtain Mzn_i . As we will see later in Section 4, c_i will become the *condition* for any dominance breaking constraints generated for Mzn . This is the main reason to eliminate only a single constraint item, as otherwise the condition in the resulting dominance breaking constraint would likely be too complex.

3.2 Extracting a CSP from a COP

Once the relaxed models are obtained, our aim is to obtain the symmetries of the original model and also those present in the relaxed models. This will allow us to reuse the symmetries found (and the associated dominance breaking constraints) for every instance of the problem.

Several methods are available to obtain the symmetries of a model, e.g. [17, 16, 4, 8]. We decided to use the method of [8], since it has been shown to have good accuracy for a wide range of benchmarks. The method first uses [7] to accurately detect a generating set for the symmetries of several small instances of the given CSP model. It then lifts these generating sets of instance symmetries to the model level, obtaining a generating set of candidate model symmetries. These candidates are filtered to eliminate those that clearly do not hold for the model, thus obtaining a generating set of likely candidates. Finally, the method marks the likely candidate generators that can be formally proved to be symmetries of the model.

The above method needs to be slightly modified to eliminate the objective function f from the models (and instances) being analysed. This is because, as mentioned before, the symmetries need to be detected for the inherent satisfaction problem since, otherwise, $f(\sigma(\theta))$ will be known to be equal — rather than better than — $f(\theta)$ for any solution θ . Chu and Stuckey [1] assumed a specification of their COP problem $P \equiv (X, D, C, f)$ where everything regarding the objective function is tucked away in f and therefore a symmetry only needs to preserve the solutions of the (X, D, C) satisfaction part. In practice, however, this is often not the case.

Example 2. Consider the following COP model of the photo placement problem from the MiniZinc distribution, where each person is assigned a different position in such a way that their combined positional preferences (who they would like to be next to) are maximised:

```

1  int: n_names;
2  int: n_prefs;
3  array[1..n_prefs,0..1] of int: prefs;
4  array[0..n_names-1] of var 0..n_names-1: pos;
5
6  constraint alldifferent(pos);
7
8  var 0..n_prefs: satisfies;
9  array[1..n_prefs] of var bool: ful;
10 constraint forall (i in 1..n_prefs) (
11     let { int: pa = prefs[i,0], int: pb = prefs[i,1] }
12     in ful[i] = (pos[pb]-pos[pa] == 1 xor pos[pa]-pos[pb] == 1) );
13 constraint sum (i in 1..n_prefs) (bool2int(ful[i])) = satisfies;
14
15 solve maximize satisfies;
```

The first three lines determine the parameters of the model (number of names of people, number of preference pairs given, and the actual array of preferences), while line 4 provides the array of decision variables assigning each position to the name of a person. Line 6 shows the only true constraint of the problem: an `alldifferent` constraint indicating the names assigned to each position need to be different. The new variables and constraints introduced from lines 8 to 13 serve only to define the objective variable `satisfies`, since they do not exclude any assignments to the decision variables in array `pos`. If we are to look for the symmetries of the satisfaction problem, we do *not* want these constraints to be included in the analysis, as they can eliminate some of the

symmetries available. This can be detrimental as it might eliminate symmetries that are likely to find better solutions.

Let us introduce a method to detect and eliminate “objective constraints” from a COP instance, that is, to obtain the satisfaction part of a COP instance by eliminating constraints whose only role is to define the objective function f without reducing the number of solutions in the instance. To do this we will use the FlatZinc version of the COP instance, obtained by compiling the MiniZinc instance into FlatZinc [10]. First, we replace the solve item with `solve satisfy`. This removes the objective variable (or, in general, the objective expression) from the file. Next, we iteratively remove variables and constraints that meet the following condition: If a variable x is used in only one constraint c , and that constraint *describes* x , then remove both c and x .

Definition 3. A constraint $c \equiv \text{con}(x_1, x_2, \dots, x_n, y)$ describes variable y if the set of solutions $\text{solns}(c)$ projected over $\{x_1, \dots, x_n\}$ is $D(x_1) \times \dots \times D(x_n)$.

Intuitively, this occurs whenever the solutions to c place no restriction on the values of the $x_1, \dots, x_n$ variables. In practice, this often corresponds to the case where y functionally depends on $x_1, \dots, x_n$, although a non-functional relationship is also possible. Clearly, whether a constraint describes a variable or not depends in general on the domains of all variables involved. This is why we use the COP instance rather than the model.

Example 3. Consider the constraint $x + y = z$, with $D(x) = D(y) = \{0, 1\}$ and $D(z) = \{0, 1, 2\}$. This constraint describes only variable z . However, if the domain was instead $D(x) = \{0, 1, 2\}$, $D(y) = \{1, 2\}$ and $D(z) = \{0, 1\}$, then the constraint would describe only variable x .

The MiniZinc-to-FlatZinc compiler can detect many cases where a constraint describes a variable, and annotates the FlatZinc with this information. For other cases, we examine the domains of the variables and the constraint itself to determine whether it describes a variable.

Example 4. Consider again the model introduced in Example 2, instantiated with a data file. The resulting FlatZinc file has many introduced variables and constraints for calculating the objective value. However, after the elimination of the objective and iteratively removing variables and constraints as described above, only the pos decision variables and the alldifferent constraint remain.

3.3 Putting it all together

The process shown in Figure 1 is now complete: starting with a COP MiniZinc model $Mzn \equiv (X, D, C, f)$ that has m constraint items $c_1, \dots, c_m$, and n MiniZinc input data files $d_1, \dots, d_n$:

1. Generate relaxed COP MiniZinc models $Mzn_1, \dots, Mzn_m$ by removing constraint items $c_1, \dots, c_m$, respectively

2. Instantiate each COP MiniZinc model (be it relaxed or not) with each of the input data files and compile to FlatZinc obtaining $Fzn_1, \dots, Fzn_n$ COP FlatZinc instances
3. For each Fzn_i COP FlatZinc instance, extract the CSP instance $FznSat_i$ and infer its generating set of symmetries S_i
4. Use the generating set of symmetries inferred for every instance $S_1, \dots, S_n$ to infer the generating set of symmetries S of either the relaxed or original model. If the latter, S are almost symmetries of Mzn ; if the former, S are symmetries of Mzn .

Note that the effect of the detected symmetries in S on any variables introduced by the flattening process can be ignored. That is, we are only interested in how the symmetry affects the variables in Mzn , because we will only apply the symmetry to those variables (see Section 4).

As mentioned before, rather than using a model-based detection approach, we could use an instance-based approach, where the symmetries are inferred only for the particular instance being considered. In this case, it would be crucial for the method to be fast, as its run-time will be an overhead over the solving time for the original COP. Clearly, complete methods (e.g. [2]) tend to be computationally expensive, even for small CSPs. Incomplete methods (e.g., [13, 14, 2, 7]) can actually be very accurate, but they can also take significant amounts of time, since the more accurate ones require building a graph G where each variable/value pair in the CSP is represented by its own vertex, in such a way that each automorphism of G corresponds to a solution symmetry of the CSP.

We attempted to use a simple and fast instance-based symmetry detection method, based on the approach of Puget [13]. Intuitively, we built a simple coloured graph where there is a vertex for each variable in the instance, and constraints are represented by additional vertices connected to the variables appearing in the constraint, in such a way that the inherent symmetry of the constraints is reflected in the graph. For example, the representation of the constraint $x \neq y$ would treat x and y identically, while the representation of $x < y$ would not. Although this method works well for certain problems, many constraints cannot be represented. For example, the `inverse(x, y)` constraint expresses a relation between the variables in x and the *values* in y ; since the values are not explicitly represented in this form of graph, the symmetry is lost.

4 Automatically Generating Dominance Breaking Constraints

Each of the symmetries and almost symmetries found in the previous step is a mapping σ that can be used for dominance breaking. As mentioned before, the general form of a dominance breaking constraint is $scond(\sigma) \rightarrow \neg ocond(\sigma)$. For a symmetry of the original model Mzn , the $scond(\sigma)$ is trivially true. For a symmetry of Mzn_i (and, thus, an almost symmetry of Mzn), $scond(\sigma)$ is $\sigma(c_i)$ where c_i is the constraint that was removed to relax the original model Mzn to obtain Mzn_i . In both cases, the $ocond(\sigma)$ for a COP with objective function f (to be minimised) is $\sigma(f) < f$. The generation of the dominance constraint requires that the mapping σ be applied to an arbitrary MiniZinc constraint or expression. This is why we restrict our symmetry detection to valuation symmetries. Our current implementation only considers *variable symmetries*; that is, permutations of the variables that induce solution symmetries.

The method to generate the dominance breaking constraints is as follows. Assume that the mapping σ is induced from a variable symmetry. We first extend σ to map every variable y in the problem that was left unchanged by σ , to a fresh variable y' . Then, for every such variable y and every constraint item c in the model, we add $\sigma(y)$ and $\sigma(c)$ to the model, respectively. Finally, we add the dominance breaking constraint $scond(\sigma) \rightarrow \neg ocond(\sigma)$. The $scond(\sigma)$ is simply the constraint removed (if any) when relaxing the problem (see Section 3.1) with occurrences of any variable y replaced by y' . The $\neg ocond(\sigma)$ is simply the constraint $\sigma(f) \geq f$, where $\sigma(f)$ is the expression f , again with occurrences of any variable y replaced by y' .

Example 5. Consider again the photo placement problem introduced in Example 2. Any permutation of the pos variables is a valuation symmetry of the satisfaction part of the problem. As an example, let us use the mapping that reverses the order of the variables pos[3], pos[4], and pos[5].

A fresh array of variables is added to represent the image of pos after the mapping. The differences from the original model are shown in bold.

```
array[0..n_names-1] of var 0..n_names-1: pos_mapped =
  [ pos[ if 3 <= i /\ i <= 5
    then 3+(5-i)
    else i endif ] | i in 0..n_names-1 ];
```

That is, pos_mapped[1] = pos[1], pos_mapped[2] = pos[2], pos_mapped[3] = pos[5], and so on.

The constraints and variables that were eliminated as part of the CSP extraction are also duplicated, but any occurrences of variables in the CSP are replaced by their mapped versions. In this example, it results in the addition of the following variables and constraints:

```
var 0..n_names-1: satisfies_mapped;
array[1..n_prefs] of var bool: ful_mapped;
constraint forall (i in 1..n_prefs) (
  let { int: pa = prefs[i,0], int: pb = prefs[i,1] }
  in ful_mapped[i] = (pos_mapped[pb]-pos_mapped[pa] == 1 xor
    pos_mapped[pa]-pos_mapped[pb] == 1) );
constraint sum (i in 1..n_prefs) (bool2int(ful_mapped[i])) =
  satisfies_mapped;
```

Finally, the dominance breaking constraint is added:

```
constraint satisfies >= satisfies_mapped;
```

While this construction can lead to a significant growth in the model, often most of the newly introduced constraint items are simple renamings of already existing items and can be immediately removed. Even those items that change may be only superficially different. For example, if the original model had a constraint alldifferent(x), then the new constraint alldifferent(x_mapped) which, although not syntactically the same, is wholly redundant.

Example 6. Most of the constraints added in Example 5 are for defining the `ful_mapped` variables. For values of i where the pa and pb values are outside the range affected by the mapping, the constraint defining `ful_mapped[i]` is identical to the one that defines `ful[i]`. As a result, the new constraint can be eliminated, and `ful_mapped[i]` made equal to `ful[i]`.

Importantly, we cannot simply add a dominance breaking constraint for every element of the symmetry group. The reasons are twofold: first, the typical size of symmetry groups is so large that to do so would incur an enormous overhead which the reduction in search space would not recoup. Second, even if the group is small, some of the associated dominance breaking constraints might by themselves already add significant overheads. This means that even restricting ourselves to the set of generators automatically inferred by our symmetry detection method might not be ideal. We would like to add only those dominance breaking constraints that will propagate efficiently and give good search space reduction. We choose the best constraints by generating a small subset of the full symmetry group, and comparing the size of the resulting constraints. While we currently do this manually, it is relatively easy to automate this process.

5 Experimental Evaluation

We have evaluated the effectiveness of the automatically generated dominance breaking constraints on some benchmark problems. For these experiments, we use the MiniZinc 2.0 compiler and Gecode 4.2.1. All times are given in seconds.

5.1 Photo Placement

We have considered two models of the photo placement problem. The first model (referred to as photo-1) is the one included in the MiniZinc distribution (and shown in Example 2). Note that we removed the manual symmetry breaking constraint from the model to get closer to the raw problem. The second model (referred to as photo-2) is the one given by Chu and Stuckey [1], and is significantly more efficient regardless of dominance breaking constraints.

For both models, after the CSP extraction the only remaining decision variables are the position variables and the only constraint is the all-different constraint. Therefore, any permutation of the position variables is a symmetry. We consider two sets of dominance mappings: all-pairs swaps and all-pairs reversals. The swaps are mappings that interchange positions i and j , for all i, j pairs. The reversals are mappings that reverse all the subsequence of positions between position i and j , for all i, j pairs. Note that the dominance constraints resulting from the reversal mappings (shown in Example 5) are the ones manually identified by Chu and Stuckey [1] for this problem.

Example 7. The following shows the duplicated variables and constraints, and added dominance breaking constraint, for the mapping that reverses positions 1 and 2.

```
array [1..n] of var 1..n : x_mapped =
  [ x [ if 1 <= i /\ i <= 2
```

```

    then 2 - i + 1
    else i endif ] | i in 1..n ];
constraint all_different(x_mapped);
constraint sum(i in 1 .. n-1) (p[x_mapped[i], x_mapped[i+1]])
- sum(i in 1 .. n-1) (p[x[i] , x[i+1]])
<= 0;

```

Let us first compare the effect on search, by showing the total number of search nodes obtained when solving different instances of each of the two models without dominance breaking, and with dominance breaking using swap mappings and reversal mappings. Note that in a branch-and-bound search, the additional constraints can *increase* the search effort if they prevent some solutions from being found early, and these solutions would have imposed constraints on the objective.

Instance	None	Swaps	Reversals
photo1-1	35,774	44,537	37,132
photo1-2	480,109	809,282	506,268
photo2-12-1	4,015,993	1,199,867	798,781
photo2-12-2	6,405,713	1,574,469	1,102,183
photo2-14-2	227,036,415	35,153,627	17,583,257

Table 1. Node counts without dominance breaking and with two different sets of mappings.

Table 1 shows how the different dominance breaking constraints affect the size of the search tree. Clearly, the reversal mappings consistently give a smaller search than the swap mappings for both models. In the first model, the dominance breaking constraints increase the node count, while in the second model they decrease it. A likely explanation for the poor performance in the first model is that the dominance breaking constraints propagate poorly and, therefore, we have the downside of excluding early solutions, but not the advantage of reducing the search tree size. We are exploring ways to automatically detect and transform the constraints to improve their effectiveness.

Next we compare the effect on running time (see Table 2). Since the reversal mappings are better than the swap mappings, we consider only those. To measure the effect of excluding redundant constraints, we manually remove the `alldifferent` constraints that are simply re-arranged versions of the original constraint. As expected, removing

Instance	None	Reversals	Reversals w/o Redundant
photo1-1	0.125	1.532	1.232
photo1-2	1.259	33.079	25.057
photo2-12-1	24.825	43.226	23.122
photo2-12-2	35.340	55.555	29.167
photo2-14-2	1626.636	1319.246	666.206

Table 2. Running time without dominance breaking, with dominance breaking, and with manual removal of some redundant constraints.

the redundant constraints reduces the running time. For those instances where the dominance breaking constraints cause a large reduction in search (all instances of photo-2), there is also a reduction in running time.

5.2 Blackhole Solitaire

In the blackhole solitaire problem, a standard deck of playing cards — minus the ace of spades — is dealt into 17 piles of three cards. These cards are played one-by-one into a discard pile, which begins with only the ace of spades. Each card played must be one higher or one lower than the current top of the discard pile, and only the top card of each pile is playable. The task is to find a sequence of plays that leads to all cards being on the discard pile.

A model for this problem (shown in Example 1) is included in the MiniZinc distribution with (a) a variable for each card, whose value is the position in the sequence it is played, and (b) a variable for each position in the sequence, whose value is the card played at that position. These variables are linked by an inverse constraint. The other two significant constraints state that cards played in succession must be compatible (their face values differ by exactly one), and that each card must be played before any card underneath it in its pile.

This model has an almost symmetry: except for the top-of-pile constraint, cards with the same face value are symmetric. Relaxing the problem to remove this constraint leads to its automatic symmetry detection, that is, finding 37 pairs of cards that can be swapped and thus used as dominance mappings.

The removed top-of-pile constraint is as follows:

```
constraint forall(i in 1..17, j in 1..2)
  ( position[layout[i,j]] < position[layout[i,j+1]] );
```

where `layout[i,j]` represents the j th card of pile i , and variable `position[card]` represents the position in the sequence that card is to be played. To compute the *scond* for the dominance breaking constraint for a single mapping, we must apply the mapping to this constraint. Since this is a satisfaction problem, the *ocond* imposes an arbitrary ordering on the variables to break the symmetry. This leads to the following dominance breaking constraint:

```
constraint
  (forall(i in 1..17, j in 1..2)
    (y_mapped[layout[i,j]] < y_mapped[layout[i,j+1]]))
  -> lex_lesseq(y, y_mapped);
```

where `y_mapped` is the image of `y` under the specific mapping.

This constraint has two problems. First, it requires the solver to support a reified version of the lexicographical ordering constraint. Second, the left hand side of the implication is too strong, causing very weak propagation.

The first problem can be fixed by observing that the `y` variables are constrained to be all-different and, therefore, the lexicographical constraint can be reduced to `y[a] < y[b]`, where `a` and `b` are the two cards being swapped by the mapping.

The second problem is more difficult to fix automatically. A single mapping affects at most two of the piles; the other piles are unchanged and so that part of the condition

is known to be true and could be eliminated from the `forall`. We have manually altered the constraint to eliminate any of these redundant parts. The resulting dominance constraints are the ones manually identified by Chu and Stuckey [1] for this problem.

Instance	None	Dominance
blackhole-9	346,973	8,221
blackhole-14	206,016,606	11,054,241
blackhole-16	> 336,752,365	4,015,958
blackhole-18	312,905	30,560
blackhole-20	916,488	62,131

Table 3. Node counts without and with dominance breaking.

Again, we first compare the effect on search reduction for several instances of the model (see Table 3). Clearly, the dominance breaking constraints cause a large reduction in search space. Without the manual modification described above, the dominance breaking constraint appears does not reduce the search space at all.

Instance	None	Dominance
blackhole-9	13.009	0.449
blackhole-14	3687.822	210.896
blackhole-16	> 7200	128.218
blackhole-18	9.123	1.138
blackhole-20	16.889	1.709

Table 4. Running time without and with dominance breaking.

Next, we compare the effect on running time (see Table 4). The running times are also greatly reduced by the dominance breaking constraints.

5.3 Talent Scheduling

The talent scheduling problem requires a set of film scenes to be arranged in some order so as to minimise actors' wages. Each scene requires a subset of actors, and actors are paid for the duration that they are on set, which is from the time of their first scene to the time of their last scene. As a result, the aim is to find a scene schedule that minimises the amount of time the actors spend waiting.

We use the model of Chu and Stuckey [1]. As with the photo placement problem, once the objective-related constraints are eliminated by the CSP extraction, the only constraint that remains is an `alldifferent` constraint on the decision variables that map scenes to schedule. Therefore, the resulting CSP has symmetries indicating that all scenes can be reordered freely.

However, the automatically generated dominance breaking constraints for these symmetries — whether they swap pairs of variables, or reverse a subsequence — are

harmful for this model, as they lead to an increase in both the search space and running time. As discussed previously, the extra constraints can interfere with the finding of solutions which would otherwise help the branch and bound search. In this case, the dominance breaking constraints also bring no benefit, a phenomenon that we will illustrate with an example.

Consider an instance where scene 1 requires actors A, B and C, while scene 2 requires only actor B. It is clear that if these two scenes are filmed at times 1 and 2, then scene 2 should come first: this means that actors A and C can arrive later and need not wait during the filming of scene 2. Any solution that begins with scene 1 and then scene 2 will be dominated by the same solution where those two scenes are swapped.

Ideally, this would be detected after scene 1 and scene 2 are assigned to time slot 1 and 2 respectively, and the search would backtrack immediately. However, this is not the case, as we can see by observing the domains of the involved variables. Consider the variable that represents the wage of actor A. Before search begins, the lower bound on their on-set time is the number of scenes they are in, and the upper bound is the total number of time slots in the schedule. Under the assignment “scene 1” then “scene 2”, their upper bound does not change, but we now know that they wait for at least one time slot so their lower bound increases slightly. Under the other assignment, “scene 2” then “scene 1”, their lower bound does not change but their upper bound decreases slightly because we know they are not on set in the first time slot.

This small change in the bounds is not sufficient to cause backtracking. The vital missing piece of information is that the rest of their schedule has exactly the same cost under both assignments. Without this, it is only at the leaves of the search tree that the dominance breaking takes effect.

6 Conclusion

Dominance relations are often present in constraint optimisation problems. We have presented an approach to automatically detecting these relations by finding the symmetries of the satisfaction core of the problem. Also, the method can find conditional dominance relations by finding the “almost symmetries” of the satisfaction core.

We have also presented a method to automatically generate dominance breaking constraints for detected relations. These constraints can significantly reduce the search effort and computation time required to solve the problems. However, adding such constraints does not always improve the model; in some cases they can add overhead without benefit, or actually enlarge the search tree.

These issues raise several avenues for further work. To improve propagation, it is important to simplify the generated constraints automatically. To reduce the running time, constraints that are wholly redundant should be eliminated, perhaps by improving the canonicalisation of the constraints so that common subexpression elimination [15] can merge them.

References

1. Geoffrey Chu and Peter J. Stuckey. A generic method for identifying and exploiting dominance relations. In Michela Milano, editor, *Principles and Practice of Constraint Program-*

- ming, volume 7514 of *Lecture Notes in Computer Science*, pages 6–22. Springer Berlin Heidelberg, 2012.
2. David A. Cohen, Peter Jeavons, Christopher Jefferson, Karen E. Petrie, and Barbara M. Smith. Symmetry definitions for constraint satisfaction problems. *Constraints*, 11(2-3):115–137, 2006.
3. Lise Getoor, Greger Ottosson, Markus Fromherz, and Björn Carlson. Effective redundant constraints for online scheduling. In *Proceedings of the Fourteenth National Conference on Artificial Intelligence*, pages 302–307, 1997.
4. Toni Mancini and Marco Cadoli. Detecting and breaking symmetries by reasoning on problem specifications. In Zucker and Saitta [18], pages 165–181.
5. Roland Martin. The Challenge of Exploiting Weak Symmetries. In Brahim Hnich, Mats Carlsson, François Fages, and Francesca Rossi, editors, *Proceedings of the International Workshop on Constraint Solving and Constraint Logic Programming*, volume 3978 of *Lecture Notes in Computer Science*, pages 149–163. Springer, 2005.
6. Christopher Mears, Maria Garcia de la Banda, Bart Demoen, and Mark Wallace. Lightweight dynamic symmetry breaking. *Constraints*, 19(3):195–242, 2014.
7. Christopher Mears, Maria Garcia de la Banda, and Mark Wallace. On implementing symmetry detection. *Constraints*, 14(4):443–477, 2009.
8. Christopher Mears, Maria Garcia de la Banda, Mark Wallace, and Bart Demoen. A novel approach for detecting symmetries in CSP models. In Laurent Perron and Michael A. Trick, editors, *CPAIOR*, volume 5015 of *LNCS*, pages 158–172. Springer, 2008.
9. P. Meseguer and C. Torras. Exploiting symmetries within constraint satisfaction search. *Artificial Intelligence*, 129(1-2):133–163, 2001.
10. N. Nethercote, P.J. Stuckey, R. Becket, S. Brand, G.J. Duck, and G. Tack. MiniZinc: Towards a standard CP modelling language. In C. Bessiere, editor, *Principles and Practice of Constraint Programming-CP*, volume 4741 of *LNCS*, pages 529–543. Springer, 2007.
11. Steven Prestwich and J. Christopher Beck. Exploiting dominance in three symmetric problems. In *Fourth International Workshop on Symmetry and Constraint Satisfaction Problems*, pages 63–70, 2004.
12. Les Proll and Barbara Smith. ILP and constraint programming approaches to a template design problem. *INFORMS Journal of Computing*, 10:10–265, 1997.
13. Jean-François Puget. Automatic detection of variable and value symmetries. In Peter van Beek, editor, *CP*, volume 3709 of *Lecture Notes in Computer Science*, pages 475–489. Springer, 2005.
14. Arathi Ramani and Igor L. Markov. Automatically exploiting symmetries in constraint programming. In Boi Faltings, Adrian Petcu, François Fages, and Francesca Rossi, editors, *CSCLP*, volume 3419 of *Lecture Notes in Computer Science*, pages 98–112. Springer, 2004.
15. Andrea Rendl. *Effective compilation of constraint models*. PhD thesis, The University of St Andrews, 2010.
16. Pierre Roy and François Pachet. Using symmetry of global constraints to speed up the resolution of constraint satisfaction problems. In *ECAI98 Workshop on Non-binary Constraints*, pages 27–33, 1998.
17. Pascal Van Hentenryck, Pierre Flener, Justin Pearson, and Magnus Ågren. Compositional derivation of symmetries for constraint satisfaction. In Zucker and Saitta [18], pages 234–247.
18. Jean-Daniel Zucker and Lorenza Saitta, editors. *Abstraction, Reformulation and Approximation, 6th International Symposium, SARA 2005, Airth Castle, Scotland, UK, July 26-29, 2005, Proceedings*, volume 3607 of *Lecture Notes in Computer Science*. Springer, 2005.